Homework 07

Use only when explicitly invoked to perform an LDA/QDA-based predictive classification analysis and write a concise report.

Use either R or Python.

Question 1: LDA

Let's start by estimating the components of LDA. From the lecture notes, for each target point x_0, we find the class label k with the largest score

$$x_0^\top \Sigma^{-1} \mu_k - \frac{1}{2}\mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k .$$

Thus, we need to estimate:

- the prior probabilities π_k;
- the mean vector (centroid) μ_k for each class; and
- the pooled covariance matrix Σ.

Use saheart.csv, the `SAheart` data from `ElemStatLearn`. The two classes are defined by `chd`, the coronary heart disease indicator. Use all nine other variables as numerical predictors, coding `famhist` as 1 for `Absent` and 2 for `Present`. The following code constructs an $n \times 9$ predictor matrix X and an n-entry response vector y, where n is the number of observations.

R | Python

```r
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd
```

a. Calculate the prior probabilities, class means, and pooled covariance matrix from these data. Use the pooled covariance denominator $n - K$ from the lecture, where $K = 2$ is the number of classes.
b. Use your estimated values to predict the label of each observation in the training data. These are in-sample fitted labels. Provide the confusion matrix, with observed classes in rows and predicted classes in columns. Check that the dimensions of your matrices and vectors match. Calculate the scores yourself for this part.
c. Perform the same LDA analysis using `MASS::lda` in R or `LinearDiscriminantAnalysis(solver="svd")` in Python, using the same numerical predictors and empirical class priors. Provide the confusion matrix. Do the results match?

ANSWER:

```r
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd
```

```r
dim(X)
```

```
[1] 462   9
```

```r
length(y)
```

```
[1] 462
```

```r
priors <- prop.table(table(y))
priors
```

```
y
        0         1
0.6536797 0.3463203
```

Priors is 0: 0.6536797, 1: 0.3463203

```r
mu0 <- colMeans(X[y == 0, ])
mu1 <- colMeans(X[y == 1, ])

mu0
```

```
      sbp    tobacco        ldl  adiposity    famhist      typea    obesity
135.460265   2.634735   4.344238  23.969106   1.317881  52.367550  25.737450
   alcohol        age
 15.931358  38.854305
```

```r
mu1
```

```
      sbp    tobacco        ldl  adiposity    famhist      typea    obesity
143.737500   5.524875   5.487938  28.120250   1.600000  54.493750  26.622937
   alcohol        age
 19.145250  50.293750
```

```r
means <- rbind(
  "chd = 0" = mu0,
  "chd = 1" = mu1
)

means
```

```
             sbp  tobacco      ldl adiposity  famhist    typea  obesity
chd = 0 135.4603 2.634735 4.344238  23.96911 1.317881 52.36755 25.73745
chd = 1 143.7375 5.524875 5.487938  28.12025 1.600000 54.49375 26.62294
          alcohol      age
chd = 0 15.93136 38.85430
chd = 1 19.14525 50.29375
```

Means are:

sbp tobacco ldl adiposity famhist typea obesity

chd = 0 135.4603 2.634735 4.344238 23.96911 1.317881 52.36755 25.73745 chd = 1 143.7375 5.524875 5.487938 28.12025 1.600000 54.49375 26.62294 alcohol age chd = 0 15.93136 38.85430 chd = 1 19.14525 50.29375

First center observations around their own class mean:

```r
X0 <- X[y == 0, ]
X1 <- X[y == 1, ]

centered0 <- sweep(
  X0,
  2,
  mu0,
  "-"
```

```
)

centered1 <- sweep(
  X1,
  2,
  mu1,
  "-"
)
```

```
n <- nrow(X)
K <- 2

Sigma <- (
  t(centered0) %*% centered0 +
  t(centered1) %*% centered1
) / (n - K)

Sigma
```

```
                sbp    tobacco         ldl  adiposity    famhist        typea
sbp       405.4347786 14.5851882  4.58124834 49.1643582 0.33698819 -15.58772153
tobacco    14.5851882 19.2425564  0.76320271  7.5381093 0.01582050  -2.05730592
ldl         4.5812483  0.7632027  4.00058129  6.0326970 0.09185024   0.34459233
adiposity  49.1643582  7.5381093  6.03269701 56.7528931 0.43280832  -5.30956633
famhist     0.3369882  0.0158205  0.09185024  0.4328083 0.22583357   0.08112007
typea     -15.5877215 -2.0573059  0.34459233 -5.3095663 0.08112007  95.56564291
obesity    18.9388430  1.8334506  2.66005159 22.7078852 0.18402776   2.64006728
alcohol    64.4003517 20.5170451 -2.53290675 16.1190253 0.76846885   7.96003164
age        95.1338895 22.7654113  6.47892049 60.5086715 0.99736251 -20.27833492
             obesity     alcohol         age
sbp       18.9388430  64.4003517  95.1338895
tobacco    1.8334506  20.5170451  22.7654113
ldl        2.6600516  -2.5329068   6.4789205
adiposity 22.7078852  16.1190253  60.5086715
famhist    0.1840278   0.7684688   0.9973625
typea      2.6400673   7.9600316 -20.2783349
obesity   17.6154236   4.6893563  15.6969995
alcohol    4.6893563 598.2766133  27.8859413
age       15.6969995  27.8859413 184.1321373
```

```
dim(Sigma)
```

```
[1] 9 9
```

And lastly, the pooled covariance matrix.

```
Sigma_inv <- solve(Sigma)
```

```
score0 <- X %*% Sigma_inv %*% mu0 -
  0.5 * as.numeric(
    t(mu0) %*% Sigma_inv %*% mu0
  ) +
  log(priors["0"])
```

```
score1 <- X %*% Sigma_inv %*% mu1 -
  0.5 * as.numeric(
    t(mu1) %*% Sigma_inv %*% mu1
  ) +
  log(priors["1"])
```

```
dim(X)
```

```
[1] 462   9
```

```
length(mu0)
```

```
[1] 9
```

```
dim(Sigma_inv)
```

```
[1] 9 9
```

```
length(score0)
```

```
[1] 462
```

```
length(score1)
```

```
[1] 462
```

Conceptually you have:

$$X: n \times 9$$
$$\Sigma^{-1}: 9 \times 9$$
$$\mu_k: 9 \times 1$$

so

$$X\Sigma^{-1}\mu_k$$

produces an (n) score vector.

```
y_pred_manual <- ifelse(
  score1 > score0,
  1,
  0
)
```

```
cm_manual <- table(
  Observed = y,
  Predicted = y_pred_manual
)

cm_manual
```

```
        Predicted
Observed   0   1
       0 258  44
       1  73  87
```

Now I compare my calculations to R's implementation:

```
library(MASS)

lda_fit <- lda(
  x = X,
```

```
  grouping = y,
  prior = as.numeric(priors)
)

lda_pred <- predict(
  lda_fit,
  X
)$class
```

```
cm_lda <- table(
  Observed = y,
  Predicted = lda_pred
)

cm_lda
```

```
        Predicted
Observed   0   1
       0 258  44
       1  73  87
```

Now check if they match:

```
all(
  y_pred_manual == as.numeric(
    as.character(lda_pred)
  )
)
```

```
[1] TRUE
```

They match, so the results did match the manually calculated LDA predictions.

Question 2: QDA and marginal screening

In the lecture, ordinary QDA could not be fitted to the handwritten digit sample because its within-class covariance matrices were singular. Let's consider selecting fewer variables and see whether this helps. For this question, use the full zip-train.csv.gz as the training data and zip-test.csv.gz as the test data, with all ten digit classes. These are the original `zip.train` and `zip.test` data from `ElemStatLearn`. Each file has no header: the first column is the digit label and the remaining 256 columns are pixel values.
Instead of using all 256 predictors, select the 40 with the largest marginal variances in the training data. Here, marginal variance means the variance of one pixel across all training observations. Use the original pixel values without standardizing them, and retain the same selected predictors in the test data.
Use `MASS::qda` in R or `QuadraticDiscriminantAnalysis(tol=1e-8)` in Python.

a. Fit QDA using these 40 predictors and report the test confusion matrix, with observed classes in rows and predicted classes in columns. Also report the test misclassification rate.
b. Does the method work? Why do you think it works or does not work?
c. Repeat the analysis using the 10 predictors with the largest marginal variances in the training data. How does the test misclassification rate compare with the 40-predictor version? Why do you think this happens?

ANSWER:

```
zip_train <- read.csv(
  "data/zip-train.csv.gz",
  header = FALSE
)

zip_test <- read.csv(
  "data/zip-test.csv.gz",
  header = FALSE
)
```

```
y_train <- zip_train[, 1]
X_train <- as.matrix(zip_train[, -1])

y_test <- zip_test[, 1]
X_test <- as.matrix(zip_test[, -1])
```

```
dim(X_train)
```

```
[1] 7291  256
```

```
dim(X_test)
```

```
[1] 2007  256
```

```
pixel_var <- apply(
  X_train,
  2,
  var
)
length(pixel_var)
```

```
[1] 256
```

```
top40 <- order(
  pixel_var,
  decreasing = TRUE
)[1:40]

top40
```

```
 [1] 230 219 105 185 121 120 204  56 235 136 169 220 213 137  76 201  27  38  69
[20]  22 200 104 214 153 152  53 107 123  60  54  57 234  92  91  75 139 154 184
[39] 168  43
```

```
pixel_var[top40]
```

```
     V231      V220      V106      V186      V122      V121      V205       V57
0.7995005 0.7786657 0.7761626 0.7760059 0.7698937 0.7651751 0.7642945 0.7634517
     V236      V137      V170      V221      V214      V138       V77      V202
0.7614990 0.7610922 0.7609557 0.7572460 0.7567441 0.7499158 0.7443015 0.7433579
      V28       V39       V70       V23      V201      V105      V215      V154
0.7427205 0.7414070 0.7394806 0.7391538 0.7385493 0.7379827 0.7365760 0.7365185
     V153       V54      V108      V124       V61       V55       V58      V235
0.7356207 0.7354114 0.7342641 0.7341245 0.7334178 0.7315470 0.7308114 0.7302140
      V93       V92       V76      V140      V155      V185      V169       V44
0.7301541 0.7284644 0.7264357 0.7255123 0.7226698 0.7214531 0.7213691 0.7213038
```

```
X_train_40 <- X_train[, top40]
X_test_40 <- X_test[, top40]
dim(X_train_40)
```

```
[1] 7291   40
```

```
dim(X_test_40)
```

```
[1] 2007   40
```

```
library(MASS)

qda40 <- qda(
  x = X_train_40,
  grouping = y_train,
  tol = 1e-8
)
```

```
pred40 <- predict(
  qda40,
  X_test_40
)$class
```

```
cm40 <- table(
  Observed = y_test,
  Predicted = pred40
)

cm40
```

```
        Predicted
Observed   0   1   2   3   4   5   6   7   8   9
       0 319   0  24   6   2   2   1   0   5   0
       1   0 238   5   0  11   0   7   0   3   0
       2   8   0 168   8   8   3   0   0   3   0
       3   5   0   4 128   0  14   0   0  15   0
       4   0   2  13   1 150   1   0   1   7  25
       5  12   0   3   5   1 131   0   0   6   2
       6   0   0   8   0   1   8 148   0   5   0
       7   1   0   6   1   6   0   0 124   2   7
       8   1   0  10  12   3   2   0   0 137   1
       9   0   0   1   2   7   1   1   4   9 152
```

The misclassification rate is:

$$\frac{\text{number incorrectly classified}}{\text{total number of observations}}$$

```
error40 <- mean(
  pred40 != y_test
)

error40
```

```
[1] 0.1554559
```

```
error40 * 100
```

```
[1] 15.54559
```

The misclassification rate is 15.546%

QDA works after reducing the predictor dimension from 256 to 40 because estimating a separate 256 x 256 covariance matrix for each class requires a very large number of parameters and can result in singular covariance matrices. Marginal screening substantially reduces the dimension and makes covariance estimation more feasible. However, the 40 selected pixels can still be highly correlated, so the covariance estimates may remain somewhat unstable.

But, overall, yes, the method works reasonably well. Using marginal screening reduces the number of predictors from 256 to 40, which makes estimating the class-specific covariance matrices required by QDA more manageable. QDA must estimate a separate covariance matrix for each digit class, and doing this with all 256 predictors can lead to singular or unstable covariance matrices. Reducing the dimension to 40 predictors helps address this problem while retaining enough information to distinguish among the digits. The resulting test misclassification rate of approximately 15.55% indicates that the model has useful predictive ability.

```
top10 <- order(
  pixel_var,
  decreasing = TRUE
)[1:10]
```

```
X_train_10 <- X_train[, top10]
X_test_10 <- X_test[, top10]
```

```
qda10 <- qda(
  x = X_train_10,
  grouping = y_train,
  tol = 1e-8
)
```

```
pred10 <- predict(
  qda10,
  X_test_10
)$class
```

```
cm10 <- table(
  Observed = y_test,
  Predicted = pred10
)

cm10
```

```
        Predicted
Observed   0   1   2   3   4   5   6   7   8   9
       0 298   0  11   1   1  16  28   4   0   0
       1   0 241  10   0   3   1   1   0   2   6
       2  19   2  95  14  14   3  22  21   5   3
       3   5   0   6 103   3  42   0   0   7   0
       4   1   5  14   0  95   2   4  15  17  47
       5  15   0   6  25   4  96   4   2   8   0
       6  41   0  15   4   0  18  78   7   5   2
       7   4   1   6   0   6   0   1 122   0   7
       8   1   2  11  36  12  11   4   1  74  14
       9   0   2   9   2  20   0   0  16  10 118
```

```
error10 <- mean(
  pred10 != y_test
)

error10
```

```
[1] 0.3423019
```

```
error10 * 100
```

```
[1] 34.23019
```

```
data.frame(
  Predictors = c(40, 10),
  Test_Misclassification_Rate = c(
    error40,
    error10
  )
)
```

```
  Predictors Test_Misclassification_Rate
1         40                   0.1554559
2         10                   0.3423019
```

Using only the 10 predictors with the largest marginal variances increased the test misclassification rate from 0.1555 (15.55%) with 40 predictors to 0.3423 (34.23%) with 10 predictors. Although reducing the number of predictors makes the class-specific covariance matrices easier and more stable to estimate, in this case the reduction removed too much useful information about the handwritten digits. The 40-predictor model therefore provides a better balance between retaining predictive information and reducing the dimensionality of the covariance estimation problem.
So fewer predictors are not automatically better. There is a bias–variance tradeoff: 10 predictors make QDA easier to estimate, but here the loss of predictive information is large enough that test performance becomes substantially worse.

Question 3: Compare LDA and QDA

In this simulation, we will study how the relative performance of LDA and QDA changes with the true covariance structure. The parameter ρ controls the distribution from which we generate the data; it is not a tuning parameter of LDA or QDA.
Consider a classification problem with two equally likely classes, labeled 1 and 2, and $p = 10$ covariates. The first two covariates have the class-conditional distributions

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \mid Y = 1 \sim N_2\left(\begin{pmatrix} -1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}\right),$$

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \mid Y = 2 \sim N_2\left(\begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}\right).$$

In both classes, the remaining eight covariates are independent standard normal variables, independent of the first two. Consider

$$\rho \in \{0, 0.2, 0.4, 0.6, 0.8, 0.9\}.$$

For each value of ρ, generate 100 training observations and 1,000 independent test observations from each class. Fit ordinary LDA and QDA using all 10 covariates and equal class priors. Estimate the class means from the training data, with one pooled covariance matrix for LDA and one full covariance matrix per class for QDA. Set a random seed and report it.

a. Generate one dataset for each value of ρ. Fit both methods and calculate their test misclassification rates. Within each setting, use the same training and test observations for the two methods.
b. At each fixed value of ρ, repeat the experiment 50 times, generating new training and test observations in each repetition. Plot average test error against ρ in one figure, with one curve for LDA and one for QDA. These curves compare performance across different data-generating distributions.
c. Compare the two methods at small and large values of ρ. How does changing the true covariance structure affect their relative performance? Explain the pattern using their covariance assumptions and the bias-variance trade-off. How can the eight noise covariates affect performance even though they contain no information about the class label?

ANSWER:

```
library(MASS)

set.seed(307)

rhos <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)
```

```
generate_data <- function(rho, n_per_class) {

  Sigma1 <- matrix(
    c(1, rho,
      rho, 1),
    nrow = 2,
    byrow = TRUE
  )

  Sigma2 <- matrix(
    c(1, -rho,
      -rho, 1),
    nrow = 2,
    byrow = TRUE
  )

  # First two predictors
  X12_class1 <- mvrnorm(
    n = n_per_class,
    mu = c(-1, 0),
    Sigma = Sigma1
  )

  X12_class2 <- mvrnorm(
    n = n_per_class,
    mu = c(1, 0),
    Sigma = Sigma2
  )

  # Eight noise predictors
  noise1 <- matrix(
    rnorm(n_per_class * 8),
    nrow = n_per_class,
    ncol = 8
  )

  noise2 <- matrix(
    rnorm(n_per_class * 8),
    nrow = n_per_class,
    ncol = 8
  )

  X1 <- cbind(X12_class1, noise1)
  X2 <- cbind(X12_class2, noise2)

  X <- rbind(X1, X2)

  y <- factor(
    c(
      rep(1, n_per_class),
      rep(2, n_per_class)
    )
  )
```

```
  list(X = X, y = y)
}
```

```
fit_models <- function(train, test) {

  lda_fit <- lda(
    x = train$X,
    grouping = train$y,
    prior = c(0.5, 0.5)
  )

  qda_fit <- qda(
    x = train$X,
    grouping = train$y,
    prior = c(0.5, 0.5)
  )

  lda_pred <- predict(
    lda_fit,
    test$X
  )$class

  qda_pred <- predict(
    qda_fit,
    test$X
  )$class

  lda_error <- mean(
    lda_pred != test$y
  )

  qda_error <- mean(
    qda_pred != test$y
  )

  c(
    LDA = lda_error,
    QDA = qda_error
  )
}
```

```
results_once <- data.frame(
  rho = rhos,
  LDA = NA,
  QDA = NA
)

for (i in seq_along(rhos)) {

  rho <- rhos[i]

  train <- generate_data(
    rho = rho,
    n_per_class = 100
  )

  test <- generate_data(
    rho = rho,
    n_per_class = 1000
  )

  errors <- fit_models(
    train,
    test
  )

  results_once$LDA[i] <- errors["LDA"]
  results_once$QDA[i] <- errors["QDA"]
}

results_once
```

```
  rho    LDA   QDA
1 0.0 0.1640 0.195
2 0.2 0.1705 0.188
3 0.4 0.1655 0.203
4 0.6 0.1725 0.182
5 0.8 0.1580 0.123
6 0.9 0.1745 0.100
```

```
set.seed(307)

results <- data.frame(
  rho = numeric(),
  repetition = integer(),
  LDA = numeric(),
  QDA = numeric()
)
```

```
for (rho in rhos) {

  for (rep in 1:50) {

    train <- generate_data(
      rho = rho,
      n_per_class = 100
    )

    test <- generate_data(
      rho = rho,
      n_per_class = 1000
    )

    errors <- fit_models(
      train,
      test
    )

    results <- rbind(
      results,
      data.frame(
        rho = rho,
        repetition = rep,
        LDA = errors["LDA"],
        QDA = errors["QDA"]
      )
    )
```

```
  }
}
```

```
head(results)
```

```
     rho repetition    LDA    QDA
LDA    0          1 0.1640 0.1950
LDA1   0          2 0.1890 0.2070
LDA2   0          3 0.1800 0.2040
LDA3   0          4 0.1645 0.1985
LDA4   0          5 0.1785 0.1935
LDA5   0          6 0.1750 0.1925
```

```
dim(results)
```

```
[1] 300   4
```

```
average_results <- aggregate(
  cbind(LDA, QDA) ~ rho,
  data = results,
  FUN = mean
)

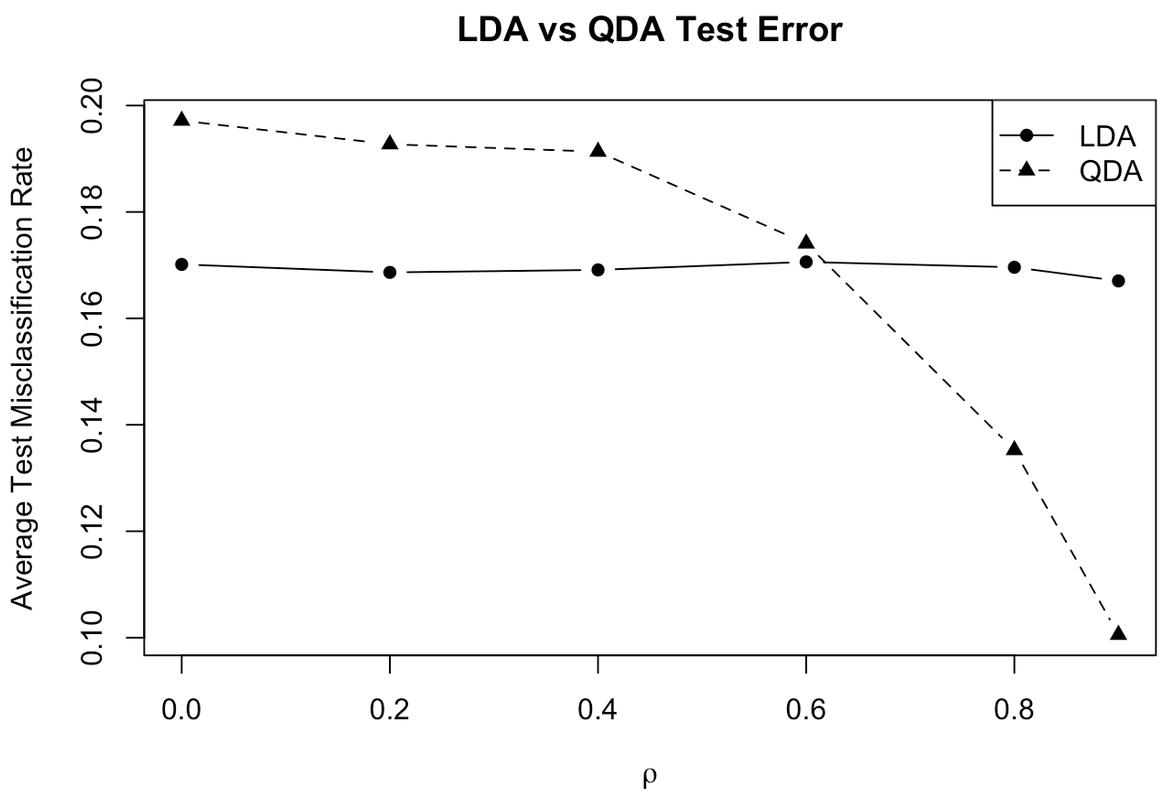
average_results
```

```
  rho     LDA     QDA
1 0.0 0.17014 0.19715
2 0.2 0.16864 0.19272
3 0.4 0.16911 0.19132
4 0.6 0.17061 0.17407
5 0.8 0.16960 0.13525
6 0.9 0.16704 0.10054
```

```
plot(
  average_results$rho,
  average_results$LDA,
  type = "b",
  pch = 16,
  ylim = range(
    average_results$LDA,
    average_results$QDA
  ),
  xlab = expression(rho),
  ylab = "Average Test Misclassification Rate",
  main = "LDA vs QDA Test Error"
)

lines(
  average_results$rho,
  average_results$QDA,
  type = "b",
  pch = 17,
  lty = 2
)

legend(
  "topright",
  legend = c("LDA", "QDA"),
  pch = c(16, 17),
  lty = c(1, 2)
)
```



When ρ=0, class 1 and class 2 covariance are the same. This is what LDA assumes. So LDA is correctly specified. QDA can also fit the data, but it unnecessarily estimates two separate covariance matrices. Therefore, we'd generally expect LDA to do better or at least similarly when ρ is small.
As ρ increases, class 1 and class 2 have very different covariance structures. LDA still tries to use one common covariance matrix while QDA doesn't. So as ρ increases, the true model increasingly favors QDA.
LDA estimates one covariance matrix that gives: less flexibility, fewer parameters, lower variance, and potentially more bias.
QDA estimates a covariance matrix for each class that gives: more flexibility, more parameters, higher variance, and potentially lower bias.

At small ρ, LDA's assumption is approximately correct, so its lower variance is beneficial.At large ρ, LDA's assumption becomes badly wrong. Its bias increases, while QDA can capture the actual class-specific covariance structures. So we'd expect QDA to become increasingly advantageous relative to LDA as ρ increases.
The last 8 predictors in both classes provide zero actual information about Y. But because you only have a finite training sample, the models don't know they're useless. For example, by random chance, the training sample might make X7 look slightly different between the two classes. The model estimates that random difference. That's estimation variance. QDA is particularly vulnerable because it estimates separate covariance matrices for each class. Adding useless predictors means it has many additional covariance parameters to estimate. So, the noise variables can hurt prediction even though they contain no genuine signal.

Question 4: Create and use an LDA/QDA analysis skill

In Homework 5, you created a skill for formatting plots. Now create a reusable skill that guides an AI agent through an LDA/QDA analysis and produces a concise statistical report. You may work with AI to design and write the skill; you do not need to write every line yourself.

a. **Define the skill.** Write a single `SKILL.md` file no larger than **5 KB (5,000 bytes)**. Define the agent's behavior, analysis guidelines, and expected report structure. Keep the skill independent of this question: provide the dataset, response, and predictors separately when invoking it. The skill may guide the agent to explore lower-dimensional representations, such as principal component analysis (PCA), or to blend the class probabilities predicted by LDA and QDA. Keep all classifiers based on LDA, QDA, or their combination. Configure the skill to load **only when you explicitly invoke it**. Use your AI tool's invocation settings, or keep the file outside automatically loaded skill folders and supply it only in an explicit request.

Your skill must require a clearly readable analysis report of **at most one printed page**, containing a figure, a table, and numerical model-fitting results. You may specify the type and style of figures and tables you want the AI to produce. All instructions in the skill must be independent of the actual dataset, its variable names, and its observed results. The report should briefly describe the candidate models, the cross-validation procedure and results, the selected model, and its estimated prediction error. It should explain the main findings and any limitations.

b. **Apply the skill.** Explicitly invoke your skill to analyze the `SAheart` data from Question 1, using the same response and predictor coding. Ask the agent to seek the best predictive performance within the LDA/QDA methods allowed by your skill. Include ordinary LDA and QDA as comparisons; the extensions above are optional. Reserve a test set before choosing models. Use the same cross-validation folds within the training data to compare candidates and select any settings. Estimate any preprocessing, including an embedding, using only the training observations in each fold. Refit the selected model and preprocessing on all training observations, then evaluate the fitted classifier once on the reserved test set. Use misclassification rate to compare performance, and report your random seed.

c. **Review and submit.** Check the agent's analysis and make any skill revisions using only training and cross-validation results. Append the final one-page analysis report and the complete contents of your `SKILL.md` to your homework report. State the file size and briefly explain how you enabled explicit invocation, including the prompt used to apply the skill. The one-page limit applies to the analysis report; the appended skill and this explanation are outside that limit.

Your skill and analysis report will be reviewed and graded by other students through peer evaluation.
ANSWER: Result: PCA-QDA with 4 components had the lowest 5-fold CV misclassification rate among 41 candidates. Its CV error was 0.246; after refitting on the training data, its one-time reserved-test error was 0.302 (35 of 116).
The analysis used seed 43261, a stratified 25% test split, and shared stratified 5-fold CV. Scaling and PCA were fit within each CV training fold. QDA covariance diagnostics were full rank.
The report includes the candidate comparison table, performance figure, results, and limitations, and was verified as one printed page.
The `lda-qda-analysis` skill is saved at `homework/week-07/SKILL.md`, outside the agent's automatically loaded `~/.copilot/skills/` directory. I invoked it explicitly by directing the agent to read that file and apply its instructions to the Question 1 data. During review, I used training-set and cross-validation results only to assess the candidate comparison and clarify the skill's compact-table guidance for tuning grids. The reserved test set was not used for model selection or skill revisions.
The skill file is **2,841 bytes**, below the 5,000-byte limit. The one-page analysis PDF is **5,656 bytes**. The prompt used to apply the skill was:

> Explicitly invoke your skill to analyze the `SAheart` data from Question 1, using the same response and predictor coding. Ask the agent to seek the best predictive performance within the LDA/QDA methods allowed by your skill. Include ordinary LDA and QDA as comparisons; the extensions above are optional. Reserve a test set before choosing models. Use the same cross-validation folds within the training data to compare candidates and select any settings. Estimate any preprocessing, including an embedding, using only the training observations in each fold. Refit the selected model and preprocessing on all training observations, then evaluate the fitted classifier once on the reserved test set. Use misclassification rate to compare performance, and report your random seed.

The selected candidate had the lowest mean cross-validation misclassification rate among the 41 settings considered. The reserved test result was obtained after model selection and was not used to revise the skill or choose the model.

SAheart classification: LDA and QDA

Seed 43261 | n=462 | p=9 | train/test=346/116 | missing predictors=0

Response: chd; all other 9 variables used as numeric predictors;

famhist coding: Absent=1, Present=2.

Class counts (0/1): overall 302/160; training 226/120; test 76/40.

Stratified 25% holdout was reserved before modeling. All candidate settings

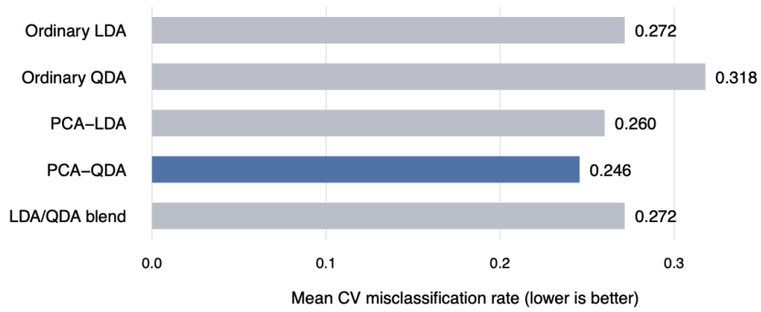
shared identical 5–fold stratified CV on training data; selection used mean

fold misclassification rate (41 settings in total).

Cross–validation comparison (training data only)

Candidate family (best setting)	Mean CV error
Ordinary LDA	0.272
Ordinary QDA	0.318
PCA–LDA (7 PCs)	0.260
PCA–QDA (4 PCs)	0.246
LDA/QDA blend (LDA weight=0.90)	0.272

Best cross–validated setting in each candidate family



Selected model: PCA–QDA (4 PCs); mean CV misclassification rate = 0.246.

Refit on all 346 training observations; one–time reserved–test error =

0.302 (35/116).

Stability: ordinary QDA covariance ranks were 9/9 and condition numbers

13.7/17.1; selected QDA ranks were 4/4 and condition numbers 3.3/2.1.

Full rank and finite condition numbers support estimability, but one split

and CV tuning leave uncertainty; these predictive associations are not causal.

Scaling and PCA were fit within folds; PCA dimension and blend weight used the same folds.

Final `SKILL.md` contents

```
---
name: lda-qda-analysis
description: Use only when explicitly invoked to perform an LDA/QDA-based predictive classification analysis and write a concise report.
---

# Intent

Produce a reproducible, concise classification analysis from the dataset, response, and predictors supplied at invocation. Keep the workflow independent of particular datasets or variable names. Use only LDA, QDA, or combinations of their predicted class probabilities.

# Analysis instructions

- Confirm the requested response and predictors. Report sample size, class counts, predictor count, missingness, and the random seed. Use a reproducible seed and stratified train/test split when feasible; document split sizes.
- Reserve the test set before modeling. Never use it for preprocessing, model selection, tuning, or revisions. Always consider ordinary LDA and ordinary QDA. Optional candidates are PCA followed by LDA or QDA, and weighted LDA/QDA probability blends.
- Compare candidates on identical cross-validation folds within the training set using misclassification rate. Fit all data-dependent preprocessing separately inside each fold, using only that fold's analysis portion; apply it unchanged to its validation portion. Select PCA dimension or blend weight by the same cross-validation procedure.
- Select the candidate with the lowest mean cross-validation misclassification rate. Report fold count and candidate results. If a candidate cannot be fit on a fold, document the failure and its implication; do not silently substitute another model.
- Refit the selected pipeline on all training data, then evaluate it once on the reserved test set. Clearly distinguish CV error from test error.
- Check covariance estimation for numerical stability, especially QDA when class sizes are small relative to predictor count. Report singularity, rank deficiency, or other material limitations. Do not make causal claims.

# Report

Create a readable report that fits on one printed page. Verify the rendered page count; shorten or reformat if needed without omitting required results. Include:

1. Seed, train/test sizes, class counts, and a brief candidate-model description.
2. A brief description of the CV design and selection metric.
3. A compact table with ordinary LDA and QDA as separate baselines; for larger tuning grids, show each candidate family's best setting and mean CV rate, and state how many settings were compared.
4. One informative, clearly labeled figure showing model performance or predictive structure.
5. The selected model, its CV error, and its one-time reserved-test error.
6. A concise interpretation and the main limitations.

Write for a statistically literate reader. Avoid unnecessary code and raw output. Use the dataset, response, and predictors supplied with the explicit request; ask for any that are missing.
```